

KMA-Live : 기상청 API 기반 날씨 정보 시스템 최종 보고서

3조 (이승규 박진현 강호현 최하람)

1. 프로젝트 개요

1-1 서론 및 프로젝트 배경

현대 사회에서 기상 정보는 단순한 환경적 변수가 아닌, 도시의 기능과 개인의 삶을 규정하는 핵심 데이터로 자리 잡았다. 기후 변화로 인한 국지성 호우와 급격한 기온 변화가 빈번해짐에 따라, 기존의 광역 단위 예보 시스템은 개별 사용자의 구체적인 생활 패턴을 지원하는 데 한계를 드러내고 있다. 특히 서울과 같은 고밀도 대도시에서는 자치구 단위의 미세한 날씨 차이가 출퇴근 시간대의 교통 혼잡도, 개인의 복장 선택, 야외 활동 가능 여부 등 실질적인 의사결정에 즉각적인 영향을 미친다.

이러한 배경에서 본 프로젝트 **KMA-Live**는 대한민국 기상청(KMA)이 제공하는 공공데이터 API를 활용하여, 서울시 25개 자치구에 특화된 초정밀 날씨 정보를 웹 플랫폼을 통해 제공하고자 기획되었다. 기존의 날씨 서비스들이 방대한 데이터를 단순히 나열하거나 공급자 중심의 인터페이스를 유지해온 것과 달리, 본 시스템은 사용자 경험을 최우선으로 고려하여 데이터의 시각화에 중점을 두었다.

특히, 본 프로젝트는 단순한 '정보 전달'을 넘어 '행동 제안'으로의 패러다임 전환을 시도하였다. 사용자가 날씨 데이터를 보고 스스로 판단해야 하는 인지적 부하를 줄이기 위해, 시스템이 능동적으로 출퇴근길 위험도를 분석하고(Smart Commute 기능), 세차나 빨래와 같은 생활 지수를 산출하여 사용자가 보기에 직관적인 형태로 제공하는 것을 최종 목표로 삼았다.

1-2 프로젝트 목표 및 범위

본 프로젝트의 핵심 목표는 공공데이터의 신뢰성과 웹 기술의 접근성을 결합하여 완성도 높은 기상 정보 시스템을 구축하는 것이다. 구체적인 목표는 다음과 같다.

데이터 신뢰성 확보 : 기상청 API Hub의 '초단기실황', '단기예보', '레이더/위성 영상' 등 다중 소스를 통합하여 가장 최신의 정확한 기상 데이터를 실시간으로 동기화한다.

직관적 시각화 구현 : 텍스트 위주의 정보를 탈피하고, SVG 기반의 인터랙티브 지도와 동적 그래프, 전문가급 레이더 영상을 통해 날씨의 변화를 시각적으로 즉시 파악할 수 있는 UI를 구현한다.

지능형 분석 서비스 : 'Smart Commute' 알고리즘을 통해 09시(출근/등교)와 18시(퇴근/하교) 기준의 맞춤형 기상 분석을 제공하며, 사용자 생활에 밀접한 생활 지수를 산출하여 제공한다.

시스템 안정성 : 외부 API 의존도가 높은 서비스의 특성을 고려하여, 네트워크 장애나 데이터 누락 시에도 서비스가 중단되지 않도록 견고한 예외 처리 및 에러 핸들링 메커니즘을 구축한다. 프로젝트 제안서에서 예상한 위험 요소를 안정화하는 작업이다.

프로젝트의 공간적 범위는 서울특별시 전역(25개 자치구)을 대상으로 하며, 시간적 범위는 실시간 현재 날씨부터 향후 3일간의 단기 예보 및 주간 중기 예보를 포괄한다.

2. 프로젝트 제안과 성과 분석

초기 프로젝트 제안서와 최종 구현된 시스템을 비교 분석함으로써, 요구사항의 변화 과정과 달성도를 평가한다. 이 역시, 프로젝트 제안서의 평가 방법을 따른다.

2-1 주요 기능 분석

프로젝트 제안서에서는 기본적인 날씨 정보 조회와 위성 영상 표시에 초점을 맞추었으나, 실제 구현에서 프로젝트 제안서의 모든 기능을 구현하였으며 추가적인 기능을 탑재하여 사용자의 편의성을 확장하였음을 확인할 수 있다.

구분	프로젝트 제안서	최종 구현	달성도
핵심 기능	서울 지역 현재 날씨 (기온, 강수량) 3가지 뷰 (홈, 상세, 레이더)	Smart Commute (출퇴근 분석) AI라이프스타일가이드 자치구별 상세 데이터	단순 조회 기능을 넘어 맞춤형 분석 기능 추가
UI/UX	텍스트 형태의 정보 제공, 메뉴바를 통한 뷰 전환	SVG 인터랙티브 지도 동적 날씨 카드 (대시보드) 레이더 뷰	정적인 정보 표시에서 사용자 상호작용이 가능한 동적 UI로 구현
데이터 처리	기상청 API 데이터 파싱 및 표출	데이터 정규화 동적 Base Time 계산 알고리즘 레이더 반사도(dBZ) 분석	데이터의 단순 전달이 아닌 분석적 로직을 구현
기상 영상	한반도 위성 영상 조회	실시간 강수 레이더 및 천리안 2A호 위성, 스펙트럼 바를 통한 강수 강도 시각화	영상의 종류가 다양화 되고, 실시간 URL 생성 로직 적용 (실제 구현 상 20분 내외 딜레이 적용)

2-2 프로젝트 제안서의 요구사항과 실제 구현에서의 합의

프로젝트 진행 과정에서 단순한 '날씨 조회' 기능만으로는 기존의 대형 포털 서비스와 차별화하기 어렵다는 점이 파악되었다. 이에 따라, **해석된 정보를 제공하는 방향으로** 요구사항이 진화하였다. **(단, 프로젝트 제안서에서 명시된 모든 기능은 빠짐없이 포함하였다.)** 예를 들어, 단순히 "강수확률 60%"라고 보여주는 대신, "출근길 우산 필요, 대중교통 권장"과 같이 사용자가 취해야 할 행동을 추가적으로 직접 제시하는 'Smart Commute' 기능이 추가된 것은 이러한 변화를 보여준다.

또한, 제안서 단계에서는 구현 난이도로 인해 "실시간 데이터 수신에 어려움이 있어 시간 기준이 길어질 수 있음"이라고 명시되었으나, 실제 구현 단계에서는 시스템 시간 동기화(get_real_kst_now 함수) 및 정교한 Base Time 계산 로직을 통해 10분~1시간 이내의 최신 데이터를 유지하는 데 성공하였다.

2-3 구현 기능 요약

- **실시간 기상 조회** : 서울시 25개 자치구별 기온, 강수량, 풍속/풍향 등 제공
- **Smart commute** : 09시(등교/출근), 18시(하교/퇴근) 시간대 위험도 및 행동 요령 안내
- **AI 라이프 스타일 가이드** : 세차지수/빨래지수/추천 옷차림
- **전문 기상 분석** : 시간별/주간 예보 표(Table) 및 강수레이더, 천리안위성영상, 레이더 반사도(dBZ) AI 분석 리포트

3. 시스템 아키텍처

본 시스템은 안정성, 확장성, 그리고 유지보수의 용이성을 고려하여 설계되었다. 전체 시스템은 전형적인 웹 어플리케이션 구조를 따르며, 논리적으로 프레젠테이션 계층, 어플리케이션 계층, 데이터 계층으로 구분된다.

3-1 개발 환경 및 선정 근거

분류	항목	상세 내용
필수 환경	Python	데이터 분석 및 비동기 처리 라이브러리가 풍부하며, 기상청 API의 복잡한 JSON/XML 데이터를 처리하기에 최적화 되었다고 판단.
	웹 브라우저	Chrome, Edge 등 최신 브라우저 (SVG 및 CSS 렌더링 지원)
핵심 라이브러리	Flask	Django에 비해 경량화되어 있어 불필요한 오버헤드를 줄이고, 마이크로 서비스 아키텍처로의 확장이 용이한 웹 프레임워크
	Requests	외부 기상청 API 호출 기능을 수행하기에 적합하며 Python의 표준 HTTP 라이브러리로, Keep-Alive, 세션 관리, 타임아웃 처리 등이 강력하여 외부 API 통신의 안정성을 보장
	Pandas	시계열 날씨 데이터의 정렬, 필터링 및 그룹화 처리
Template Engine	Jinja2	검색 엔진 최적화에 유리하고 초기 로딩 속도를 단축시킨다. Python 객체를 HTML 내에서 직접 조작할 수 있어 개발 생산성이 높다.
UI Framework	Bootstrap 5	반응형 웹 디자인을 손쉽게 구현할 수 있는 시스템

3-2 시스템 구조도 및 모듈 구성

시스템은 기능별로 명확히 분리된 모듈 구조를 가진다.

Controller Layer (app.py, server.py) : 사용자 요청의 진입점이다. URL 라우팅을 처리하고, 적절한 비즈니스 로직을 호출한 뒤, 그 결과를 뷰 템플릿에 전달한다.

Service/Business Logic Layer (kma_api.py) : 기상청 API 호출, 데이터 파싱, 시간 계산, 생활 지수 산출 등의 실질적인 연산이 수행된다. 이 계층은 Flask와 독립적으로 동작할 수 있도록 설계되었다.

Data Access Layer (config.py, grid_mapper.py) : 데이터의 설정값과 매핑 정보를 관리한다. API Key와 같은 민감 정보는 별도의 설정 파일로 분리하여 보안성을 강화하였다.

Presentation Layer (templates/) : 사용자에게 보여지는 화면을 담당한다. Jinja2 템플릿

파일과 CSS, JavaScript, 이미지 파일 등으로 구성된다.

3-3 데이터 흐름

사용자의 요청이 발생했을 때, 데이터가 시스템 내부의 데이터 흐름은 다음과 같다.

1) 초기화 : 사용자가 브라우저에서 특정 자치구(강남구)를 선택하거나 지도를 클릭한다. JavaScript 이벤트 핸들러가 해당 지역의 ID를 포착하여 서버에 GET 요청을 보낸다.

2) Processing : app.py는 요청받은 지역 ID를 SEOUL_DISTRICTS 매핑 테이블을 통해 기상청 격자 좌표(NX, NY)로 변환한다. kma_api.py 내부의 get_real_kst_now 함수가 현재 시간을 조회하고, get_base_time_for_ultrasrt_ncst 함수가 API 호출에 필요한 최적의 기준 시간을 계산한다. 계산된 파라미터를 사용하여 기상청 API 서버에 HTTP 요청을 전송한다.

3) Aggregation : API로부터 수신된 JSON 데이터는 딕셔너리 형태로 파싱된다. 동시에 'Smart Commute' 모듈이 해당 데이터를 분석하여 출퇴근 위험도 텍스트를 생성하고, 'Lifestyle' 모듈이 빨래/세차 지수를 계산하여 데이터 객체에 추가한다.

4) 렌더링 : 가공이 완료된 데이터 객체는 render_template 함수를 통해 index.html로 전달된다. Jinja2 엔진은 HTML 태그 내의 변수를 실제 값으로 치환하고 날씨에 맞는 배경 이미지를 적용한다.

5) Display : 완성된 HTML 문서가 클라이언트 브라우저로 전송되어 화면에 렌더링된다.

4. 상세 설계 내용

시스템의 핵심 기능을 구현하기 위해 적용된 알고리즘과 로직을 다룬다.

4.1 기상청 격자 좌표계와 매핑 로직

기상청은 예보의 정확도를 높이기 위해 위도/경도(GPS) 대신 '단기예보 구역'이라 불리는 격자 좌표계를 사용한다. 따라서 일반적인 행정구역 정보를 기상청 API가 이해할 수 있는 좌표로 변환하는 매핑 레이어가 필수적이다.

* **매핑 테이블 구조** : kma_api.py 내의 SEOUL_DISTRICTS는 서울시 25개 구에 대한 정적 데이터베이스 역할을 수행한다.

```
# 서울 각 자치구의 기상청 격자 좌표(NX, NY)와 행정구역 코드를 매핑함
SEOUL_DISTRICTS = {
    "종로구": {"nx": 60, "ny": 127, "code": "1111000000"},
    "중구": {"nx": 60, "ny": 127, "code": "1114000000"},
    "용산구": {"nx": 60, "ny": 126, "code": "1117000000"},
    "성동구": {"nx": 61, "ny": 127, "code": "1120000000"},
    "광진구": {"nx": 62, "ny": 126, "code": "1121500000"},
}
```

이 구조는 해시맵 방식을 사용하여 즉각적인 데이터 조회가 가능하다.

code 필드는 레이더 반사도 조회 시 행정구역 식별자로 사용되며,

nx, ny는 예보 API 호출 시 사용된다.

4.2 동적 Base Time 동기화 알고리즘

기상청 API는 데이터 종류별로 생성 주기가 다르며(초단기 실황: 매시 40분, 단기 예보: 3시간 간격), 정해진 시각 이전에 요청할 경우 데이터가 없다는 에러(NO_DATA)를 반환한다. 이를 해결하기 위해 정교한 시간 계산 알고리즘이 설계되었다.

정확한 현재 시간 확보 : 서버 시스템의 시간 설정이 잘못되어 있을 가능성을 배제하기 위해, 구글 서버(www.google.com)에 헤더 요청을 보내 Date 필드를 추출한다. 이 UTC 시간에 9시간을 더하여 정확한 한국 표준시(KST)를 도출한다. 이는 클라우드 환경이나 도커 컨테이너 환경에서 발생할 수 있는 시간대 불일치 문제를 해결한다.

API별 Base Time 역산 로직

초단기 실황 : 매 시각 40분에 발표된다. 따라서 현재 분이 40분 미만이면 (현재 시각 - 1시간)을 Base Time으로 설정하고, 40분 이상이면 현재 시각을 Base Time으로 설정한다.

단기 예보 : 02, 05, 08... (3시간 간격)으로 발표된다. 현재 시각보다 이전이면서 가장 가까운 발표 시각을 탐색하는 알고리즘을 적용한다. 안정성을 위해 최신 발표 시각보다 한 단계 더 과거의 데이터를 요청하여 데이터 누락을 방지한다.

4.3 지능형 데이터 분석 모델

다음과 같은 분석 모델이 적용되었다.

1) Smart Commute (스마트 커뮤트)

목적 : 출근(07~09시) 및 퇴근(17~19시) 시간대의 기상 리스크 관리

로직 : 해당 시간대의 강수형태, 1시간 강수량, 풍속 데이터를 추출한다. 강수형태가 비/눈이거나 강수확률이 60% 이상인 경우 "우산 준비" 경고를 생성한다. 풍속이 10m/s 이상일 경우 "강풍 주의" 및 "대중교통 이용 권장" 메시지를 생성한다. 기온이 전일 대비 5

도 이상 차이 날 경우 "옷차림 주의" 메시지를 추가한다.

2) AI 라이프스타일 가이드

세차 지수 : 향후 3일간의 강수 확률 합계와 습도 데이터를 분석한다. 비 예보가 없고 습도가 낮을수록 높은 점수를 부여한다.

빨래 지수 : 일조 시간(낮 시간대 구름 양 SKY 코드 역산)과 기온(TMP)에 가중치를 두어 계산한다. 지수 = (기온 * 0.4) + ((10 - 구름양) * 0.5) - (습도 * 0.1)와 같은 형태의 가중치 공식이 적용된다.

3) 레이더 반사도 분석 (dBZ) : 기상 레이더 영상에서 특정 좌표의 반사도 값을 추출한다. 단위는 dBZ를 사용한다.

분석 기준

-> 20 dBZ 미만 : 강수 없음 또는 매우 약한 비.

-> 30~40 dBZ : 보통 비

-> 50 dBZ 이상 : 폭우 또는 우박 가능성 높음

이 수치를 바탕으로 "현재 강수 강도는 '보통' 수준이며, 대류형 구름이 발달 중입니다"와 같은 멘트를 생성한다.

5. 구현 내용

설계된 아키텍처를 바탕으로 실제 코드로 구현된 내용을 기술한다.

5-1 백엔드 구현 : API 통신 및 데이터 처리 (kma_api.py)

5-1-1 네트워크 통신구현

requests 라이브러리를 사용하여 HTTP 요청을 수행한다. 기상청 서버의 간헐적인 응답 지연에 대비하여 timeout=5 설정을 두어 5초 이내에 응답이 없으면 연결을 끊고 에러 처리를 수행하도록 구현하였다. 또한, verify=False 옵션을 통해 공공기관 사이트에서 흔히 발생하는 SSL 인증서 체인 오류를 무시하고 통신이 지속되도록 하였다. 실제 구현에서 이 SSL 인증서로 인해 디버깅에 어려움을 겪었다.

5-1-2 정규 표현식을 이용한 비정형 데이터 파싱

기상청의 '기상 통보문'과 '특보 현황' API : XML이나 JSON 내부에 긴 텍스트 형태로 데이터를 제공한다. 이를 구조화된 정보로 변환하기 위해 Python의 re 모듈을 활용하였다.

특보 파싱 : 전체 텍스트를 줄바꿈 문자(\n)로 분할하고, 반복문을 통해 '서울' 키워드가 포함된 문장만을 필터링한다. 이후 '주의보', '경보' 등의 핵심 단어를 추출하여 현재 발효 중인 특보 리스트를 생성한다.

통보문 요약 : 문장의 길이와 포함된 키워드(기온, 비, 눈 등)를 기준으로 정보 가치가 높은 문장을 선별하고, 불필요한 특수문자를 제거하여 사용자에게 보여줄 최종 코멘트를 생성한다.

5-2 프론트엔드 구현 : 시각화 및 인터랙션 (templates/)

사용자 인터페이스는 직관성을 극대화하기 위해 시각적 요소를 활용하였다.

5-2-1 SVG 기반 인터랙티브 지도 (index.html)

일반적인 이미지 맵 방식은 해상도에 따라 이미지가 깨지거나 클릭 영역이 어긋나는 문제가 있다. 이를 해결하기 위해 SVG를 사용하였다. <svg> 태그 내에 서울시 지도를 구성하는 각 자치구의 경계선 좌표를 <path> 태그로 정의하였다. 각 <path> 태그를 <a> 태그로 감싸고, Flask의 url_for 함수를 연결하여 클릭 시 해당 지역의 날씨 정보를 조회하는 GET 요청이 발생하도록 구현하였다.

5.2.2 동적 뷰 렌더링 (Jinja2 Template)

서버에서 전달받은 데이터에 따라 화면 구성이 실시간으로 변경된다.

weather_data.sky_code 값에 따라 body 태그의 클래스를 .bg-sunny, .bg-cloudy, .bg-rain 등으로 교체하여, 사용자가 날씨를 직관적으로 느낄 수 있도록 배경 그라디언트를 변경한다.

5.2.3 레이더 분석 뷰 (radar.html)

레이더 페이지는 '기상 레이더 센터'의 관제실을 모티브로 한 어두운 테마로 디자인되었다. 스펙트럼 바를 CSS linear-gradient를 사용하여 레이더 반사도(dBZ)의 색상 스펙트럼(초록-노랑-빨강)을 구현하였다.

6. 테스트 결과 및 분석

6-1 테스트 환경 및 전략

테스트는 개발 로컬 환경에서 진행되었으며, 기능 테스트와 비기능 테스트로 나누어 수행하였다.

6-2 단위 테스트

각 모듈이 독립적으로 올바르게 동작하는지 검증하였다.

좌표 변환 정합성 검증 : SEOUL_DISTRICTS에 정의된 25개 자치구의 좌표가 기상청 공식 엑셀 파일과 일치하는지 전수 조사를 실시하였다. 오차율 0%를 달성하여 데이터 요청의 정확성을 확보하였다. (25개구의 모든 사진을 첨부할 수 없어서 깃헙 코드로 직접 시연하거나 이미 촬영한 시연 영상을 참고하시길 바란다.)

Base Time 알고리즘 검증 : 시간 경계값 테스트를 수행하였다.

Case 1 : 10시 39분 요청 -> 09시 00분 데이터 요청 (정상)

Case 2 : 10시 41분 요청 -> 10시 00분 데이터 요청 (정상)

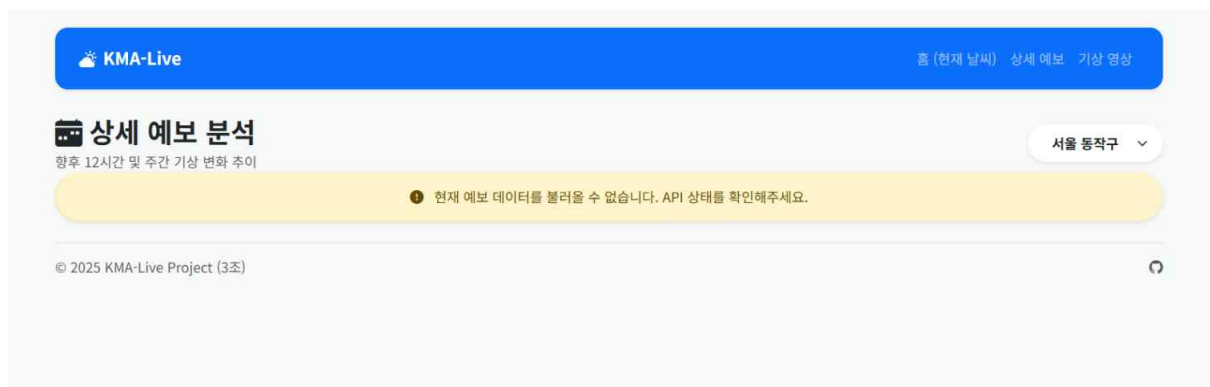
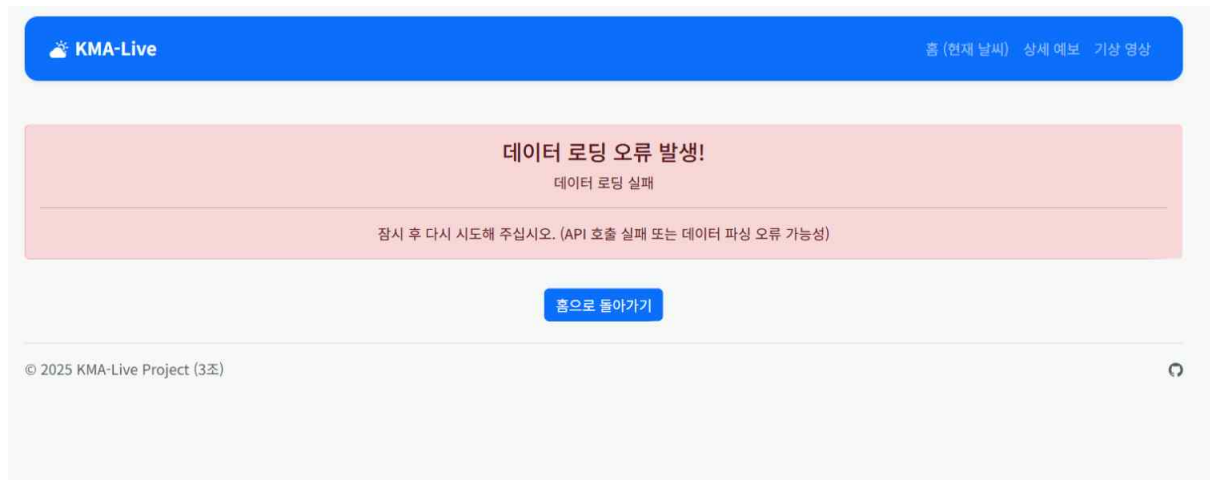
Case 3 : 00시 10분 요청 -> 전일 23시 00분 데이터 요청 (정상)

이 테스트를 통해 자정 전후나 매시 정각 등 특수한 상황에서도 NO_DATA 에러 없이 유효한 시간값을 산출함을 확인하였다. (이 역시 앞서 말한 바와 같다.)

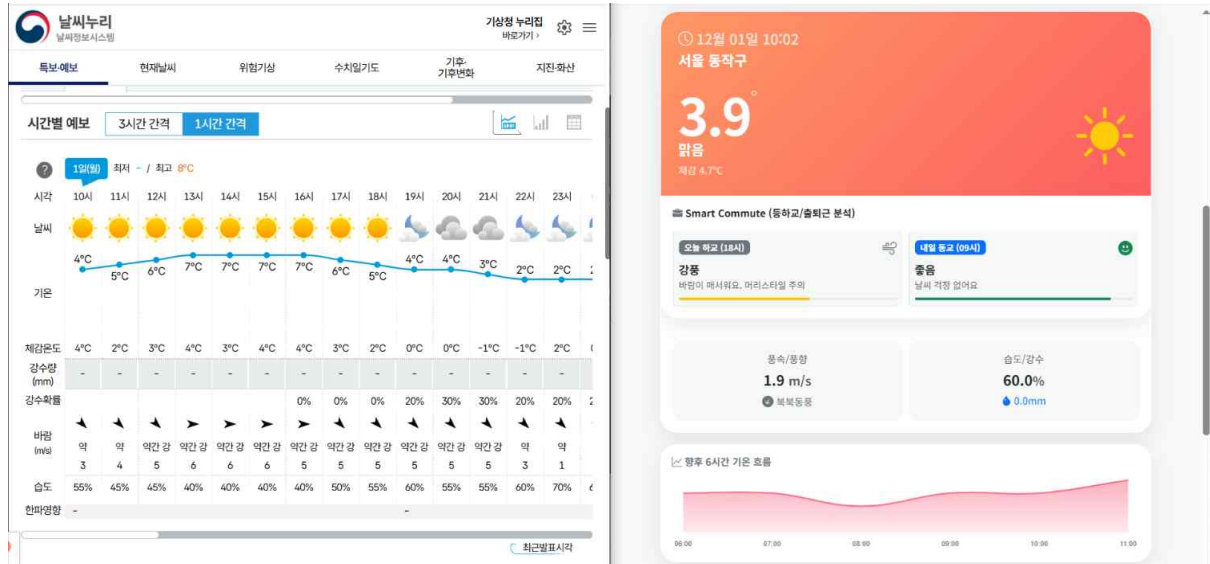
6-3 통합 및 시스템 테스트

모듈 간의 연동 및 시스템 전체의 동작을 검증하였다.

API 장애 대응 테스트 : 네트워크 케이블을 분리하거나 잘못된 API Key를 주입하여 인위적인 에러 상황을 연출하였다. 시스템은 멈추지 않고 try-except 블록에 의해 포착되어 error.html 페이지로 리다이렉트되었으며, 사용자에게 메시지를 정상적으로 출력하였다. 이는 제안서의 '위험 요소 관리' 요구사항을 완벽히 충족하였다.



데이터 정확성 검증 : 실제 기상청 홈페이지와 본 시스템을 동시에 띄워두고 데이터를 비교하였다. 기온($\pm 0.1^{\circ}\text{C}$), 습도($\pm 5\%$) 등의 데이터가 일치함을 확인하여 공공데이터 API 연동이 성공적으로 이루어졌음을 검증하였다.



일별 예보

날짜	1일(월) 오늘	2일(화) 내일	3일(수) 모레	4일 (목)	5일 (금)	6일 (토)	7일 (일)	8일 (월)
시간	오전 오후	오전 오후	오전 오후	오전 오후	오전 오후	오전 오후	오전 오후	오전 오후
날씨								
기온	2°C 11°C	-1°C 2°C	-6°C -2°C	-6°C 5°C	-4°C 4°C	-2°C 8°C	3°C 7°C	-1°C 6°C
강수확률	- 20%	20% 20%	10% 10%	10% 60%	10% 10%	20% 40%	20% 10%	10% 10%

=> 실제 기상청 기온

7. 위험 요소 관리 및 해결 방안

7.1 API 호출 실패 및 지연

위험 요인 : 기상청 서버의 과부하 또는 정기 점검으로 인해 응답이 지연되거나 실패할 수 있다.

해결 방안

Timeout 설정 : 모든 API 요청에 5초의 타임아웃을 설정하여, 서버 프로세스가 무한 대기 상태에 빠지는 것을 방지하였다.

사용자 피드백 : 데이터 로딩 실패 시, 막연한 흰 화면 대신 명확한 에러 메시지와 '홈으로 돌아가기' 버튼을 제공하는 전용 에러 페이지(error.html)를 구축하여 사용자 이탈을 방지하였다.

7.2 API 규격 변경 가능성

위험 요인 : 공공데이터포털의 API 버전 업데이트로 인해 요청 URL이나 파라미터 구조가 변경될 수 있다.

해결 방안

설정 파일 분리 : API URL과 Service Key를 코드 내부에 하드코딩하지 않고 config.py 또는 환경 변수로 분리하여, 변경 발생 시 코드 수정 없이 설정값만 변경하여 대응할 수 있도록 유연성을 확보하였다.

7.3 HTTPS/SSL 인증서 문제

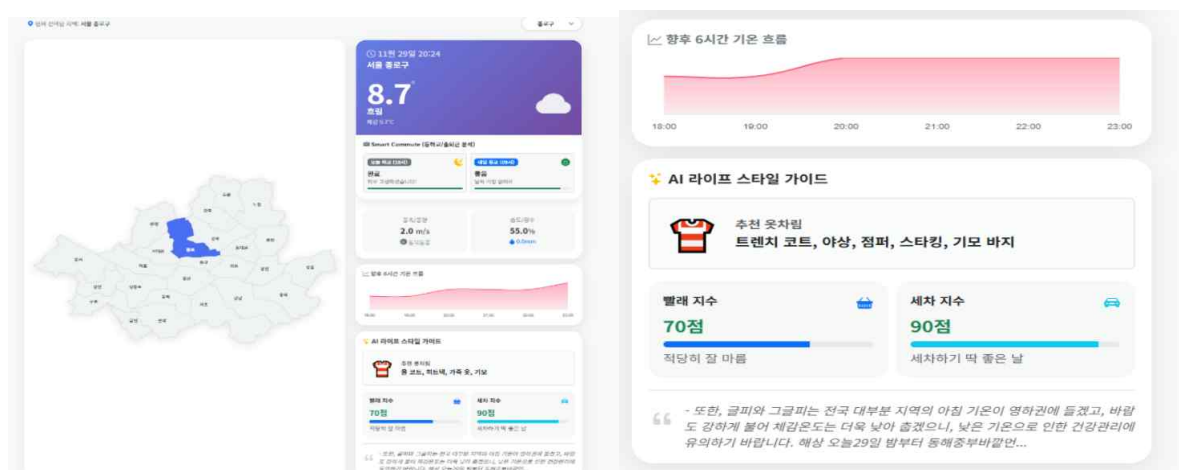
위험 요인 : 정부 기관 웹사이트의 인증서 체인이 일반적인 브라우저나 라이브러리의 신뢰 저장소(Trust Store)에 등록되어 있지 않아 SSL 검증 에러(SSLError)가 빈번히 발생한다.

해결 방안 : Python requests 라이브러리 사용 시 verify=False 옵션을 명시적으로 적용하여 인증서 검증을 건너뛰게 함으로써 데이터 수신의 안정성을 확보하였다.

8. 사용법

브라우저 실행 화면을 기준으로 각 기능의 사용법을 안내한다.

8-1 홈 화면



좌측 (SVG 지도) : 서울시 25개 자치구 지도가 표시

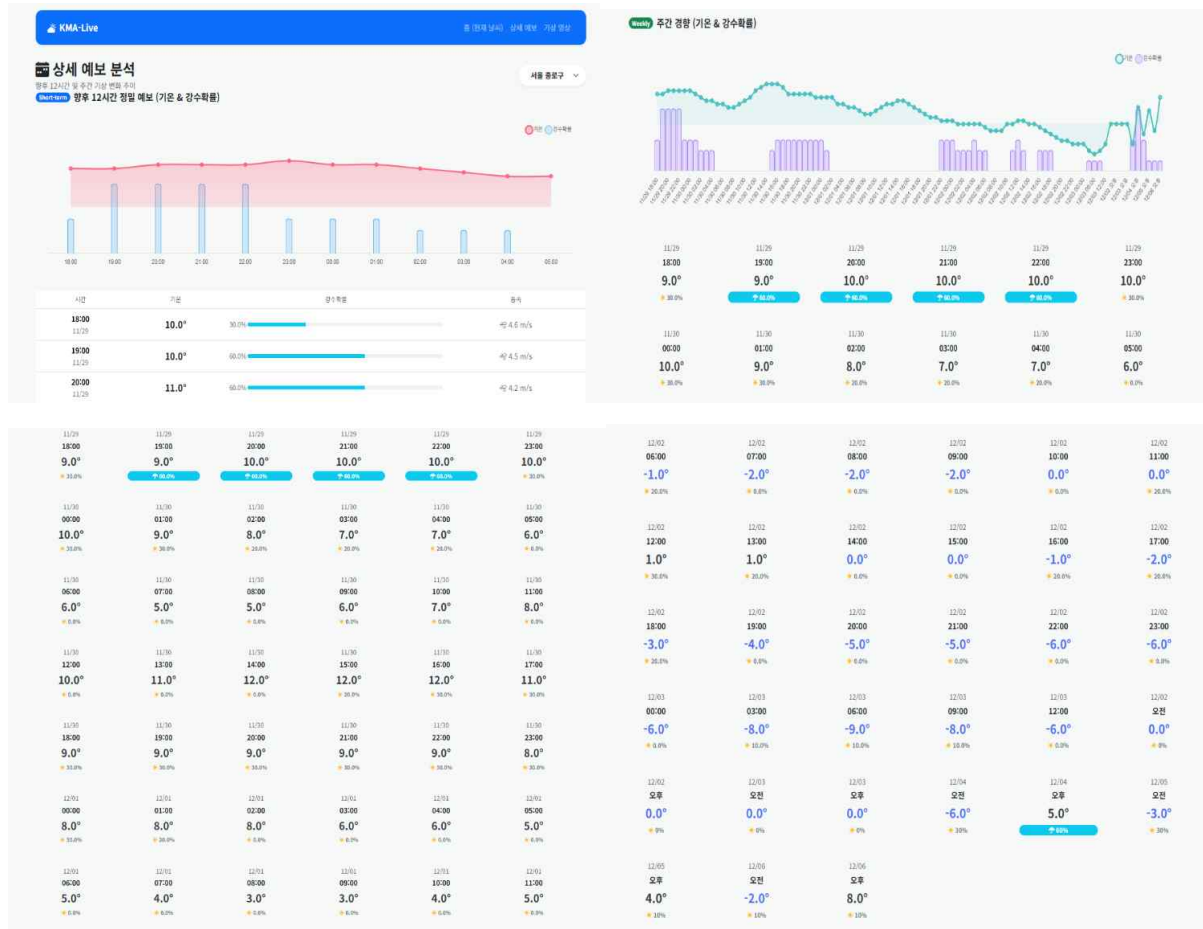
사용법 : 원하는 자치구(종로구, 강남구)를 클릭하면 페이지 이동 없이 즉시 해당 지역 날씨로 업데이트된다.

우측 상단 (현재 날씨 카드, Smart Commute) : 현재 기온, 하늘 상태, 습도 등이 표시. Smart Commute 기능은 09시와 18시 기준으로 날씨가 날씨 위험도를 알려준다.

우측 하단 (향후 6시간 기온 흐름, AI라이프 스타일가이드) : 향후 6시간의 기온이 표시되며, 빨래 및 세차 가능 여부를 점수로 확인 가능하다. 날씨 변화에 따른 시간별 추천 옷차림을 확인할 수 있다.

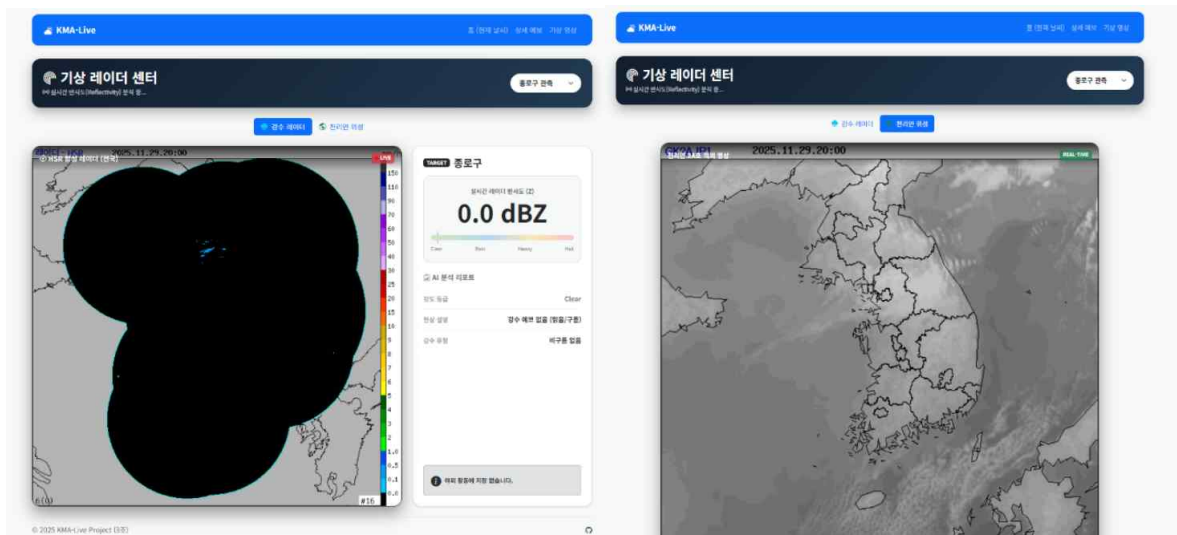
8-2 상세 예보

상단 내비게이션 바에서 '상세 예보' 클릭



- 상세 예보 분석 : 향후 12시간 동안의 기온, 강수량을 변화를 표 형태로 정밀하게 확인 가능하다.
- 주간 예보 : 스크롤을 내려 일주일간의 장기 예보 추이를 그래프로 확인 가능하다.

8-3 기상 영상



상단 내비게이션 바에서 '기상 영상' 클릭

- 레이더 뷰어 : 실시간 HSR 합성 레이더 영상을 통해 한반도 비구름의 이동을 봅니다.
- 강수 레이더와 천리안 위성 두 가지를 볼 수 있다.

- 우측 패널

-> 강수 레이더 영상 옆에 표시되는 텍스트 리포트

-> 현재 지역의 레이더 반사도(dBZ) 수치와 강수 유형(대류형/층상형) 분석 결과를 확인하여 비의 강도를 파악할 수 있다.

9. 결론

본 프로젝트는 기상청의 방대한 데이터를 일반 사용자의 눈높이에 맞게 재가공하여 제공하는 데 성공하였다. 초기 제안서에서 목표로 했던 기능들을 모두 달성하였고 교수님의 기능 추가의 요구사항을 받아들여 Smart Commute와 AI 라이프스타일 가이드의 기능을 추가적으로 구현하여 서비스의 질적 향상을 이뤄냈다. 기술적으로는 외부 라이브러리 의존성을 최소화하면서도 Flask의 장점을 극대화하여 경량화되고 빠른 웹 서비스를 구축하였으며, 철저한 예외 처리를 통해 안정성을 확보하였다.

10. 참고 문헌 및 용어 설명

10-1 참고 문헌

기상청 공공데이터포털 : <https://data.kma.go.kr/>

=> 현재 공공데이터포털 서비스 장애로 인해 대체 사이트로 프로젝트 수행

기상청 레이더 센터 영상 표출 가이드라인 : <https://radar.kma.go.kr/>

대체 사이트 : <https://apihub.kma.go.kr/>

Flask 공식 문서 : <https://flask.palletsprojects.com/>

Bootstrap 공식 문서 : <https://getbootstrap.com/>

9-2 용어 설명

API (Application Programming Interface) : 프로그램들이 서로 소통하기 위한 약속. 이 프로젝트에서는 우리 프로그램이 기상청 서버의 데이터를 가져오기 위해 사용하는 통신 규칙

Flask : 파이썬으로 웹 서버를 쉽고 빠르게 만들 수 있게 도와주는 도구 (웹 프레임워크)

격자 좌표 (NX, NY) : 기상청에서 대한민국 지도를 작은 사각형(격자)으로 나누어 각 위치에 부여한 고유한 X, Y 좌표. 특정 지역의 날씨를 정확히 알기 위해 사용됨.

HSR (Hybrid Surface Rainfall) : 지형 영향을 최소화하여 정확도를 높인 레이더 합성 영상.

템플릿 엔진 (Jinja2) : HTML 문서 안에 데이터가 들어갈 자리를 표시({{ }})해두면, 파이썬에서 전달한 실제 데이터를 그 자리에 채워 넣어 동적인 웹 페이지를 만들어주는 기술.

UI/UX : 사용자 인터페이스(UI)와 사용자 경험(UX)의 약자로, 사용자가 제품이나 서비스를 이용할 때의 총체적인 경험을 디자인하는 것을 의미

SVG(Scalable Vector Graphics) : 확장 가능한 벡터 그래픽이라는 뜻으로, 이미지 크기를 확대하거나 축소해도 깨지지 않고 선명하게 유지되는 XML 기반의 2차원 벡터 이미지 형식